

GH-600 — Cheat Sheet de repaso rápido

Resumen ejecutivo de las 6 secciones. Para repaso pre-examen.

Fecha objetivo: ~31 mayo 2026

Sección 1: Arquitectura agente y SDLC (15-20%)

- El agente es un **contribuidor junior**: su output (PRs) pasa code review, nunca merge directo a main
 - Ciclo fundamental: **Plan** → **Act** → **Evaluate** (planificar, actuar, evaluar)
 - **Separar planificación de ejecución**: el humano aprueba el plan ANTES de que el agente ejecute
 - **Autonomía proporcional al riesgo**: tareas de alto impacto → más supervisión; typos → alta autonomía
 - **GitHub = sistema de registro y plano de control** de toda actividad del agente
 - **Observabilidad completa**: qué hizo, por qué, cuál fue el resultado (PRs, commits, logs de Actions, artefactos)
 - Antipatrón más peligroso: **sobre-autonomía** (agente con permisos de merge sin supervisión)
 - Guardrails principales: **branch protection** + **required reviews** + **CODEOWNERS** + **status checks** + **rulesets**
-

Sección 2: Herramientas y entorno (20-25%) ⚠

- **Herramienta (tool)** = capacidad externa que el agente invoca (leer archivos, ejecutar terminal, APIs, crear PRs)
- Sin herramientas → chatbot; **con herramientas** → **actor autónomo**
- **4 categorías de herramientas**: integradas (built-in), extensión (MCP), flujo de trabajo (Actions), contexto (instrucciones)
- **copilot-instructions.md** = instrucciones de **COMPORTAMIENTO** (convenciones, restricciones, estilo)
- **copilot-setup-steps.yml** = configuración de **ENTORNO** (instalar deps, preparar BD, pre-tareas); se ejecuta ANTES del agente en la **VM sandbox efímera**
- **.github/agents/*.md** = agentes especializados en VS Code con herramientas y restricciones propias (frontmatter YAML: **name**, **tools**, **description**)
- **MCP (Model Context Protocol)** = "USB-C para agentes de IA"; protocolo abierto para conectar con herramientas externas
- Transportes MCP: **stdio** (local) y **SSE/HTTP** (remoto)
- Servidor MCP remoto oficial de GitHub: <https://api.github.com/mcp> (autenticación con PAT o GitHub App token)
- Config MCP en VS Code: **.vscode/mcp.json** (nivel workspace) o **settings.json** (nivel usuario)
- **MCP Allowlists**: control a nivel de **organización** — lista de servidores permitidos; modo estricto (bloquea no listados) o permisivo
- **Registros MCP**: directorios centralizados para descubrir/compartir servidores MCP (versionado semántico)
- Credenciales MCP: **nunca hardcodear** → usar **`${env:VARIABLE}`**

- Modelo de confianza en 3 capas: **Organización** → **Repositorio** → **Agente individual**
 - El Copilot coding agent **nunca puede hacer merge directo** a la rama principal (por diseño)
-

Sección 3: Memoria, estado y ejecución (10-15%)

- **3 tipos de memoria:** corto plazo (sesión), largo plazo (Copilot Memory + instructions), externa (archivos/DB/APIs)
 - **Copilot Memory:** 2 niveles — **repositorio** (Business/Enterprise) y **usuario** (Pro/Pro+); **auto-eliminación a los 28 días sin uso**
 - **Instrucciones jerárquicas:** `.github/copilot-instructions.md` (global) → `*.instructions.md` (directorio); NO expiran
 - **Persistencia de estado vía Git:** commits = checkpoints; PR descriptions = resumen acumulativo; issue comments = historial
 - **Deriva de contexto** (context drift): el agente pierde alineación con la tarea original por overflow, acumulación de ruido o sesgo de recencia
 - Corrección de drift: **anclaje al objetivo, checkpoints de validación, descomposición en sub-tareas**
 - Reanudación sin repetición: leer commits → reconstruir contexto → continuar desde el último punto válido
 - Jerarquía de precedencia: **prompt actual** > **.instructions.md local** > **copilot-instructions.md** > **Copilot Memory** > **inferencia del código**
-

Sección 4: Evaluación, errores y ajuste (15-20%)

- Definir criterios de éxito **ANTES** de ejecutar (no después)
 - **Señales cuantitativas:** CI pass rate (>95%), approval rate (>80%), merge rate (>90%), cobertura de tests (>80%)
 - **Señales cualitativas:** alineación arquitectónica, legibilidad, manejo de edge cases, scope adecuado
 - **4 categorías de causa raíz:** **Razonamiento** (lógica, hallucination), **Herramientas** (API incorrecta, parámetros mal), **Contexto** (info faltante/obsoleta/contradictoria), **Ambiente** (permisos, deps, timeouts)
 - Ciclo continuo: **Evaluar** → **Diagnosticar** → **Ajustar** → **Re-evaluar**
 - **3 ejes de ajuste:**
 - **Instrucciones:** `copilot-instructions.md`, `*.instructions.md`, `*.agent.md`, workflows explícitos
 - **Memoria:** Copilot Memory (facts persistentes), actualizar instrucciones, acceso a docs
 - **Herramientas:** configurar MCP servers, controlar tools por agente, políticas org
 - **applyTo** en frontmatter YAML de `.instructions.md` para instrucciones específicas por patrón de archivo
 - **Hallucination** = el agente inventa APIs/funciones que no existen → verificar contra documentación real
-

Sección 5: Orquestación multiagente (15-20%) ⚠

- **4 patrones de orquestación:**
 - **Fan-out (paralelo):** múltiples agentes en tareas independientes → el más natural para Copilot
 - **Pipeline (secuencial):** agente pasa resultado al siguiente en cadena
 - **Orquestador (central):** agente coordinador descompone y delega
 - **Peer Review:** un agente codea, otro revisa el PR
- Aislamiento GitHub-nativo: **ramas separadas + PRs independientes + CI aislado**
- **3 tipos de conflictos:** cambios superpuestos, esfuerzo duplicado, salidas contradictorias
- Resolución: rebase (minor) → merge manual → arbitraje humano → re-ejecución
- **3 estados de fallo:** ejecución fallida, ejecución parcial, ejecución atascada
- **Patrón Circuit Breaker:** detener re-ejecución tras N fallos consecutivos → notificar al equipo
- **PR description como documento de handoff:** contexto, decisiones, cambios, dependencias
- Commits como **logs de decisión:** [copilot] tipo: descripción
- **3 pilares de observabilidad multiagente:** trazabilidad, registro, artefactos
- Lifecycle del agente: **Creación** → **Configuración** → **Activo** → **Actualización** → **Retiro**
- Actualizaciones aplican en la **próxima ejecución** (no afectan sesiones en curso)
- Al retirar un agente: toda la historia Git/PR/issues se **preserva permanentemente**

Sección 6: Protección y responsabilidad (10-15%)

- **3 dimensiones de riesgo** (independientes): operacional, seguridad, compliance
- **4 niveles de autonomía:** totalmente autónomo, semi-autónomo, supervisado, restringido (read-only)
- **Semi-autónomo = modo estándar** del Copilot coding agent (crea PR, espera revisión humana)
- **4 capas de protección:** Prevención (allowlists, firewall, tokens) → Detección/bloqueo (branch rules, SAST, secret scanning) → Supervisión humana (reviews, CODEOWNERS, environment gates) → Auditoría (Git history, Actions logs, audit log)
- **Principio de mínimo privilegio:** tokens con scope mínimo (**contents:read**, **pull-requests:write**); nunca **repo:admin**
- **Agent Firewall:** default-deny para conexiones de red del agente en la nube
- **Paradoja de approval fatigue:** demasiadas aprobaciones → revisores aprueban sin leer → MENOR seguridad
- Acciones irreversibles necesitan: **2+ aprobaciones + periodo de gracia + plan de rollback**
- IA responsable: seguridad del contenido, transparencia, trazabilidad, monitoreo de anomalías

Conceptos transversales

Concepto	Aplicación
Mínimo privilegio	Permisos del agente, tokens, acceso a repos, MCP
Observabilidad	PRs, commits, Actions logs, session traces, audit log
Human-in-the-loop	PR reviews, environment gates, plan approval, CODEOWNERS

Concepto	Aplicación
Git como protocolo de sincronización	Estado, progreso, decisiones → todo vive en Git
Autonomía proporcional al riesgo	Bajo riesgo → autónomo; alto → supervisado/restringido
Iteración continua	Evaluar → diagnosticar → ajustar → re-evaluar

Archivos clave de GitHub para agentes

Archivo	Propósito	Ubicación
<code>.github/copilot-instructions.md</code>	Instrucciones de comportamiento del agente (repo-wide)	Repo
<code>*.instructions.md</code>	Instrucciones por directorio/tipo de archivo	Cualquier dir
<code>.github/copilot-setup-steps.yml</code>	Configuración de entorno de la sandbox (pre-ejecución)	Repo
<code>.github/agents/*.agent.md</code>	Definición de agentes especializados (VS Code)	Repo
<code>.vscode/mcp.json</code>	Configuración de servidores MCP (workspace)	Workspace
<code>.github/CODEOWNERS</code>	Enrutamiento automático de reviews por ruta	Repo
Rulesets	Políticas granulares de ramas (org o repo)	GitHub UI

Acrónimos y términos

Término	Definición
MCP	Model Context Protocol — protocolo abierto para conectar agentes con herramientas externas
SDLC	Software Development Life Cycle
SAST	Static Application Security Testing
PAT	Personal Access Token
HITL	Human-in-the-Loop — humano en el ciclo de decisión
PoLP	Principle of Least Privilege — mínimo privilegio
SSE	Server-Sent Events (transporte MCP remoto)
stdio	Standard I/O (transporte MCP local)
Context drift	Deriva de contexto — agente pierde alineación con objetivo original
Hallucination	Agente inventa APIs/funciones/datos que no existen

Término	Definición
Fan-out	Patrón de ejecución paralela de múltiples agentes
Circuit breaker	Patrón que detiene re-ejecución tras fallos consecutivos
Allowlist	Lista de permitidos (ej. servidores MCP aprobados por la org)
Rulesets	Reglas de repositorio más granulares que branch protection
Copilot Memory	Memoria persistente con auto-eliminación a 28 días sin uso